Lab Manual Programs

https://gemini.google.com/app/3a8e7ce39c2aaf4b

User prompt: Give program along with sample input output alone for the all experiment present in the lab manual Give it properly without layout mistakes Give experiments wise

Response: **Experiment 1: Installation and Setup of Data Analysis Tools**

Program:

```
# Import NumPy Library
import numpy as np
# Import Pandas Library
import pandas as pd
# Import Matplotlib library
import matplotlib.pyplot as plt
# Import Seaborn library
import seaborn as sns
# Import Folium library
import folium
# Display success message
print("All required libraries are installed successfully.")
```

Expected Output:

```
All required libraries are installed successfully.
```

Experiment 2: Exploratory Data Analysis on a Real-World Dataset

Program:

```
# Import the Pandas library
import pandas as pd
# Load the Titanic dataset
data = pd.read_csv("titanic.csv")
# Display the first five rows of the dataset
print("First Five Records")
print(data.head())
# Display basic information about the dataset
print("\nDataset Information")
print(data.info())
# Display the total number of missing values in each column
print("\nMissing Values")
print(data.isnull().sum())
# Display summary statistics of numerical columns
print("\nSummary Statistics")
print(data.describe())
# Display the names of all columns
print("\nColumn Names")
print(data.columns)
```

Expected Output:

```
First Five Records
   PassengerId  Survived  Pclass  ...     Fare Cabin  Embarked
0            1         0       3  ...   7.2500   NaN         S
1            2         1       1  ...  71.2833   C85         C
...

Dataset Information
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 12 columns):
...
```

```
Missing Values
PassengerId    0
Survived       0
...
Cabin          7
Embarked       0
dtype: int64

Summary Statistics
       PassengerId   Survived     Pclass        Age      SibSp      Parch       Fare
count    10.000000  10.000000  10.000000   9.000000  10.000000  10.000000  10.000000
mean      5.500000   0.500000   2.300000  28.111111   0.700000   0.300000  27.020820
...

Column Names
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
```

Experiment 3: Working with NumPy Arrays and Pandas DataFrames

Program:

```
# Import NumPy Library
import numpy as np
# Import Pandas library
import pandas as pd

# Create a NumPy array
numbers = np.array([10, 20, 30, 40, 50])
print("NumPy Array")
print(numbers)

# Add 5 to each element in the array
print("\nArray after adding 5")
print(numbers + 5)

# Create a dictionary
student = {
    "Name": ["Arun", "Bala", "Charan"],
    "Age": [20, 21, 22],
    "Marks": [85, 90, 88]
}
# Convert dictionary into DataFrame
df = pd.DataFrame(student)
print("\nStudent DataFrame")
print(df)

# Display only the Name column
print("\nStudent Names")
print(df["Name"])

# Add a new column
df["Grade"] = ["A", "A+", "A"]
print("\nUpdated DataFrame")
print(df)
```

Expected Output:

```
NumPy Array
[10 20 30 40 50]

Array after adding 5
[15 25 35 45 55]
```

```
Student DataFrame
     Name  Age  Marks
0    Arun   20     85
1    Bala   21     90
2  Charan   22     88

Student Names
0      Arun
1      Bala
2    Charan
Name: Name, dtype: object

Updated DataFrame
     Name  Age  Marks Grade
0    Arun   20     85     A
1    Bala   21     90    A+
2  Charan   22     88     A
```

Experiment 4: Data Cleaning and Preprocessing

Program:

```
# Import the Pandas Library
import pandas as pd
# Load the Titanic dataset
data = pd.read_csv("titanic.csv")

print("Original Dataset")
print(data.head())

print("\nMissing Values Before Cleaning")
print(data.isnull().sum())

# Fill missing values in the Age column with the mean age
data["Age"] = data["Age"].fillna(data["Age"].mean())

# Remove duplicate rows
data = data.drop_duplicates()

print("\nMissing Values After Cleaning")
print(data.isnull().sum())

print("\nCleaned Dataset")
print(data.head())
```

Expected Output:

```
Original Dataset
   PassengerId  Survived  Pclass  ...     Fare Cabin  Embarked
0            1         0       3  ...   7.2500   NaN         S
...

Missing Values Before Cleaning
PassengerId    0
Survived       0
...
Age            1
...
Cabin          7
Embarked       0
dtype: int64

Missing Values After Cleaning
```

```
PassengerId    0
Survived       0
...
Age            0
...
Cabin          7
Embarked       0
dtype: int64

Cleaned Dataset
   PassengerId  Survived  Pclass  ...     Fare Cabin  Embarked
0            1         0       3  ...   7.2500   NaN         S
...
```

Experiment 5: Basic Data Visualization using Matplotlib

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("daily_sales_2025_v2.csv")
data["Date"] = pd.to_datetime(data["Date"])

print("First Five Records:")
print(data.head())

data["Month"] = data["Date"].dt.month_name()
month_order = [
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December"
]

monthly_sales = (
    data.groupby("Month")["Sales"]
    .sum()
    .reindex(month_order)
)

# Line Chart
plt.figure(figsize=(10, 5))
plt.plot(monthly_sales.index, monthly_sales.values, marker="o", linewidth=2)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales (₹)")
plt.xticks(rotation=45)
plt.grid(True)
plt.show()

# Bar Chart
plt.figure(figsize=(10, 5))
plt.bar(monthly_sales.index, monthly_sales.values)
plt.title("Monthly Sales Comparison")
plt.xlabel("Month")
plt.ylabel("Sales (₹)")
plt.xticks(rotation=45)
plt.show()

# Pie Chart
plt.figure(figsize=(8, 8))
plt.pie(monthly_sales.values, labels=monthly_sales.index, autopct="%1.1f%%", startangle=90)
plt.title("Monthly Sales Distribution")
plt.show()
```

```
# Histogram
frequency, bins, patches = plt.hist(data["Sales"], bins=15, edgecolor="black", alpha=0.6)
bin_centers = (bins[:-1] + bins[1:]) / 2
plt.plot(bin_centers, frequency, marker="o", linewidth=2, label="Frequency Polygon")
plt.title("Distribution of Daily Sales")
plt.xlabel("Daily Sales (₹)")
plt.ylabel("Frequency")
plt.grid(True)
plt.legend()
plt.show()

# Area Chart
plt.figure(figsize=(10, 5))
plt.plot(monthly_sales.index, monthly_sales.values, marker="o", linewidth=2)
plt.fill_between(range(len(monthly_sales)), monthly_sales.values, alpha=0.3)
plt.title("Monthly Sales Area Chart")
plt.xlabel("Month")
plt.ylabel("Sales (₹)")
plt.xticks(range(len(monthly_sales)), monthly_sales.index, rotation=45)
plt.grid(True)
plt.show()

print("\nSummary Statistics of Daily Sales:")
print(data["Sales"].describe())
```

Expected Output:

```
First Five Records:
        Date  Sales  Profit  Orders
0 2025-01-01  15154    2441      75
1 2025-01-02  15253    1620      33
...

Summary Statistics of Daily Sales:
count      365.000000
mean     14992.131587
std       1507.658547
min      12039.000000
25%      13635.000000
50%      14956.000000
75%      16302.000000
max      18024.000000
Name: Sales, dtype: float64
```

(Graphs generated: Line chart, Bar chart, Pie chart, Histogram with Frequency Polygon, Area chart)

Experiment 6: Bivariate Data Analysis

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("notebooks/student_performance.csv")
print(data)

plt.figure(figsize=(6, 4))
plt.scatter(data["Hours_Studied"], data["Marks"])
plt.title("Hours Studied vs Marks")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.show()
```

Expected Output:

```
  Student  Hours_Studied  Marks
0    Arun              2     35
1    Bala              3     45
2  Charan              4     50
3  Deepak              5     60
4   Eswar              6     70
5  Farhan              7     80
6   Gokul              8     90
7    Hari              9     95
```

(Graph generated: Scatter plot showing Hours Studied vs Marks)

Experiment 7: Aggregation, Grouping, and Pivot Tables

Program:

```
import pandas as pd

data = pd.read_csv("employee_dataset_100_records.csv")
print("Employee Dataset:")
print(data.head())

department_group = data.groupby("Department")

average_salary = department_group["Salary"].mean()
print("\nAverage Salary by Department:")
print(average_salary)

total_salary = department_group["Salary"].sum()
print("\nTotal Salary by Department:")
print(total_salary)

employee_count = department_group["Employee_ID"].count()
print("\nNumber of Employees by Department:")
print(employee_count)

average_performance = department_group["Performance_Score"].mean()
print("\nAverage Performance by Department:")
print(average_performance)

summary = department_group["Salary"].agg(["count", "mean", "min", "max", "sum"])
print("\nSalary Summary:")
print(summary)

pivot_table = pd.pivot_table(
    data,
    values="Salary",
    index="Department",
    columns="Gender",
    aggfunc="mean"
)
print("\nAverage Salary by Department and Gender:")
print(pivot_table)
```

Expected Output:

```
Employee Dataset:
  Employee_ID Employee_Name         Department  Gender    Location  Salary  Performance_Score
0      EMP001    Aarav Iyer  AI & Data Science    Male     Chennai   68500                4.3
...

Average Salary by Department:
Department
AI & Data Science    62325.0
...
```

```
Total Salary by Department:
...

Number of Employees by Department:
...

Average Performance by Department:
...

Salary Summary:
...

Average Salary by Department and Gender:
Gender               Female          Male
Department
AI & Data Science  65214.29  60769.23
...
```

Experiment 8: Time Series Analysis and Visualization

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("daily_sales_2025_v2.csv")
data["Date"] = pd.to_datetime(data["Date"])
data = data.sort_values("Date")
data.set_index("Date", inplace=True)

# Daily Sales Trend
plt.figure(figsize=(12, 5))
plt.plot(data.index, data["Sales"], linewidth=1)
plt.title("Daily Sales Trend - 2025")
plt.xlabel("Date")
plt.ylabel("Sales (₹)")
plt.grid(True)
plt.show()

# Calculate 7-Day Moving Average
data["Moving_Average"] = data["Sales"].rolling(window=7).mean()

# Plot Sales and Moving Average
plt.figure(figsize=(12, 5))
plt.plot(data.index, data["Sales"], alpha=0.5, label="Daily Sales")
plt.plot(data.index, data["Moving_Average"], linewidth=2, label="7-Day Moving Average")
plt.title("Daily Sales and 7-Day Moving Average")
plt.xlabel("Date")
plt.ylabel("Sales (₹)")
plt.legend()
plt.grid(True)
plt.show()

# Monthly Sales
monthly_sales = data["Sales"].resample("ME").sum()
print("\nMonthly Sales:")
print(monthly_sales)

# Monthly Sales Visualization
plt.figure(figsize=(10, 5))
plt.plot(monthly_sales.index, monthly_sales.values, marker="o")
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
```

```
plt.ylabel("Total Sales (₹)")
plt.grid(True)
plt.show()
```

Expected Output:

```
Monthly Sales:
Date
2025-01-31    126480
2025-02-28    127420
...
Freq: ME, Name: Sales, dtype: int64
```

(Graphs generated: Daily Sales Trend, Daily Sales and 7-Day Moving Average, Monthly Sales Trend)

Experiment 9: Geographical Data Visualization

Program:

```
import pandas as pd
import folium

data = pd.read_csv("india_cities.csv")
print("First Five Records:")
print(data.head())

india_map = folium.Map(location=[22.5, 78.9], zoom_start=5)

for _, row in data.iterrows():
    popup_text = (
        f"City: {row['City']}<br>"
        f"State: {row['State']}<br>"
        f"Population: {row['Population']:,}"
    )
    folium.Marker(
        location=[row["Latitude"], row["Longitude"]],
        popup=popup_text,
        tooltip=row["City"]
    ).add_to(india_map)

india_map

# Circle Marker Version
population_map = folium.Map(location=[22.5, 78.9], zoom_start=5)
for _, row in data.iterrows():
    folium.CircleMarker(
        location=[row["Latitude"], row["Longitude"]],
        radius=max(4, row["Population"]/1000000),
        popup=row["City"],
        tooltip=row["City"],
        fill=True
    ).add_to(population_map)

population_map
```

Expected Output:

```
First Five Records:
...
```

(Output contains an interactive map of India showing the locations of the cities with markers and sized circle markers)

Experiment 10: Cartographic Visualization

Program:

```
import pandas as pd
import folium

state_data = pd.read_csv("state_sales.csv")
print(state_data.head())

geojson_file = "india_states.geojson"

india_map = folium.Map(location=[22.5, 78.9], zoom_start=5)

folium.Choropleth(
    geo_data=geojson_file,
    data=state_data,
    columns=["State", "Sales"],
    key_on="feature.properties.ST_NM",
    fill_opacity=0.7,
    line_opacity=0.2,
    legend_name="State-wise Sales"
).add_to(india_map)

folium.LayerControl().add_to(india_map)

india_map
```

Expected Output: *(Output contains a choropleth map of India where states are shaded according to their sales values)*

Experiment 11: EDA on Wine Quality Dataset

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = pd.read_csv("winequality-red.csv", sep=";")

print("First Five Records:")
print(data.head())

print("\nDataset Shape:")
print(data.shape)

print("\nData Types:")
print(data.dtypes)

print("\nMissing Values:")
print(data.isnull().sum())

print("\nDescriptive Statistics:")
print(data.describe())

# Distribution of Wine Quality
plt.figure(figsize=(8, 5))
sns.countplot(data=data, x="quality")
plt.title("Distribution of Wine Quality")
plt.xlabel("Quality Score")
plt.ylabel("Number of Wines")
plt.show()

# Alcohol Distribution
plt.figure(figsize=(8, 5))
plt.hist(data["alcohol"], bins=15, edgecolor="black")
plt.title("Distribution of Alcohol Content")
```

```
plt.xlabel("Alcohol (%)")
plt.ylabel("Frequency")
plt.show()

# Box Plot of Alcohol by Quality
plt.figure(figsize=(8, 5))
sns.boxplot(data=data, x="quality", y="alcohol")
plt.title("Alcohol Content by Wine Quality")
plt.xlabel("Quality")
plt.ylabel("Alcohol (%)")
plt.show()

# Volatile Acidity by Quality
plt.figure(figsize=(8, 5))
sns.boxplot(data=data, x="quality", y="volatile acidity")
plt.title("Volatile Acidity by Wine Quality")
plt.xlabel("Quality")
plt.ylabel("Volatile Acidity")
plt.show()
```

Expected Output: *(Text output for head, shape, dtypes, null values, and summary statistics, followed by Quality Distribution Countplot, Alcohol Content Histogram, and Boxplots)*

Experiment 12: Correlation Heatmap and Multivariate Analysis

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = pd.read_csv("winequality-red.csv", sep=";")

correlation_matrix = data.corr(numeric_only=True)
print("Correlation Matrix:")
print(correlation_matrix)

# Correlation Heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap of Wine Quality Dataset")
plt.show()

# Pair Plot
selected_columns = ["alcohol", "volatile acidity", "citric acid", "sulphates", "quality"]
sns.pairplot(data[selected_columns])
plt.show()

# Alcohol vs Quality
plt.figure(figsize=(8, 5))
sns.scatterplot(data=data, x="alcohol", y="quality")
plt.title("Alcohol vs Wine Quality")
plt.xlabel("Alcohol (%)")
plt.ylabel("Quality")
plt.show()

# Alcohol vs Quality with Regression Line
plt.figure(figsize=(8, 5))
sns.regplot(data=data, x="alcohol", y="quality")
plt.title("Relationship Between Alcohol and Quality")
plt.xlabel("Alcohol (%)")
plt.ylabel("Quality")
plt.show()
```

```
# Sulphates vs Quality
plt.figure(figsize=(8, 5))
sns.scatterplot(data=data, x="sulphates", y="quality")
plt.title("Sulphates vs Wine Quality")
plt.xlabel("Sulphates")
plt.ylabel("Quality")
plt.show()
```

Expected Output:

```
Correlation Matrix:
                      fixed acidity  volatile acidity  citric acid  ...
fixed acidity              1.000000         -0.256131     0.671703  ...
volatile acidity          -0.256131          1.000000    -0.552496  ...
...
```

(Graphs generated: Heatmap of the correlation matrix, Pairplot of selected columns, and Scatter/Regression plots for Quality vs Alcohol/Sulphates)
